Oliver Greenwood, 7932Z

Whisperwise: Emergent Communication in Multi-Agent Reinforcement Learning for Complex Tasks

Computer Science Tripos – Part II
Queens' College

2026

Declaration

I, the candidate for Part II of the Computer Science Tripos with Blind Grading Number 7932Z, hereby declare that this report and the work described in it are my own work, unaided except as may be specified below, and that the report does not contain material that has already been used to any substantial extent for a comparable purpose. In preparation of this report, I adhered to the Department of Computer Science and Technology AI Policy. [The project required the approved use of [insert name of technology] and such use is acknowledged in the text at the relevant sections.] [I am content for my report to be made available to the students and staff of the University.]

Signed: Oliver Greenwood
Date: April 5, 2026

Proforma

Candidate number:	7932Z
Project Title:	**Whisperwise: Emergent Communication in Multi-Agent Reinforcement Learning for Complex Tasks**
Examination:	**Computer Science Tripos – Part II, 2026**
Word Count:	⟨WORD COUNT⟩[1]
Code Line Count:	⟨LINE COUNT⟩[2]
Project Originator:	Oliver GReenwood
Project Supervisor:	Ian Lewis

Original Aims of the Project

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Fusce ac turpis quis ligula lacinia aliquet. Mauris ipsum. Nulla metus metus, ullamcorper vel, tincidunt sed, euismod in, nibh.

Work Completed

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Fusce ac turpis quis ligula lacinia aliquet. Mauris ipsum. Nulla metus metus, ullamcorper vel, tincidunt sed, euismod in, nibh.

Special Difficulties

None.

[1] This word count was computed using `texcount`.
[2] This code line count was computed using `cloc` (excluding autogenerated test output).

Contents

1 Introduction **1**

1.1 Project Summary/Motivation 1

1.2 Background Work 1

2 Preparation **2**

2.1 Project Overview 2

2.1.1 Simulation 2

2.1.2 Stations 2

2.1.3 Agents 2

2.2 Reinforcement Learning 3

2.2.1 Reward Function 3

2.2.2 Multi-Agent Proximal Policy Optimisation(MAPPO) 4

2.3 Protocols 6

2.3.1 No Communication 6

2.3.2 On-Policy Communication 6

2.3.3 On-Policy Communication using Codebook 6

2.3.4 AI Mother Tongue 6

2.4 Requirements Analysis 8

2.4.1 Success Criteria 8

2.4.2 Extension 9

2.4.3 MoSCoW Analysis 9

2.5 System Design & Tool Selection 9

2.5.1 Software Development Model 9

2.5.2 Tool Selection & Justification 9

2.6 Starting Point 10

3 Implementation **11**

3.1 Implementation Overview 11

3.2 System Layout 11

3.3 Repository Overview 12

3.4 Implementation Approach 12

3.4.1 Unit Testing 12

3.4.2 Documentation 12

3.5 Simulation 12

3.5.1 Environment Logic 12

3.5.2 Observations 12

3.5.3 Rewards 12

3.6 PPO Architecture 12

3.6.1 Actor-Critic Method 12

3.6.2 Vectorization and Scaling 12

3.7 Communication Channel 12

3.7.1 Continuous Baseline 12

3.7.2 Discrete VQ-VAE 12

3.7.3 Linguistic Metrics Implementation 12

3.8 Summary 12

4 Evaluation **13**

4.1 Hmm 13

5 Conclusions **14**

5.1 yep 14

Bibliography **15**

A Proposal **16**

Chapter 1

Introduction

1.1 Project Summary/Motivation

Limited bandwidth in communication channels - discrete symbols reduce bandwidth
Look at how communication in RL is currently done - issues?, is it used? - does it assume telepathy?

Communication Vaccum Equilibrium: No need for communication, so doesnt bother using it.

LLMs - stuck using human language, even though human language is designed for biological and social needs.

1.2 Background Work

Previous work:

Obvious direct related work:
AI Mother Tongue - implementing this VQ-VAE idea into basic 2 agent tasks
VQ-VAE - Paper from a few years ago
TRPO -¿ PPO -¿ MAPPO
Reinforcement learning paper
Current communication in RL

Other important works:
VQ-VAE for compression - see info theory
Some form of analysis of communication using information theory

Chapter 2

Preparation

Well-motivated project with success criteria well-justified. Challenging and well-presented background covering Comp Sci topics beyond Part IB. Good requirements analysis, justified selection of suitable tools, good engineering approach.

2.1 Project Overview

2.1.1 Simulation

To evaluate communication, we investigate through the following cooperative game.

There is an MxN board. The game lasts for T timesteps and at every timestep, every agent simultaneously does one action. The goal of the game is for the agents to cooperate and complete a required recipe.

2.1.2 Stations

Stations are dotted randomly across the map and are immovable throughout the course of the game. Each station has an assigned ingredient. The station can either be of type 'pick up' or 'drop off'. A 'pick up' station only allows the agents to take ingredients from them. A 'drop off' station only allows the agents to give the station the assigned ingredient.

2.1.3 Agents

Each agent is two sets of ingredients - an input set and an output set. Their input set determines which ingredients the agents need to collect. Once they have all of their ingredients, they get removed from their inventory and replaced with their output ingredients. The agents can collect ingredients from 'pick up' stations or other agents. The agents can drop off their ingredients at 'drop off' stations.

One important feature of this game is the agents have limited range of sight and can only view the nearest entity at any given timestep. This converts this game from a Markov Desicion Process to a Partially Observable Markov Decision Process. The agents must work together to discover the required stations and use them to complete recipes.

At every timestep, every agents takes a single action. Agents have five possible actions: move north, move south, move west, move east and interact. The first four of which move the agent one step in the given direction. If agent A executes the 'Interact' action:

1. If visible agent B has a required item, place in A's inventory, and remove from B's inventory.
2. If visible 'pick up' station has a required item, place in A's inventory.
3. If visible 'drop off' station wants an item we have, remove from A's inventory.

2.2 Reinforcement Learning

Reinforcement Learning (RL) is different to classical supervised and unsupervised learning as we do not have all the data before training. We must build environments for agents, who take actions and recieve feedback from these actions. The agents must learn to maximise the total reward collected over time.

Classic RL is modelled through Markov Decision Processes (MDPs), but to investigate communication, we must limit the information every agent has so they must communicate missing information with one another. Therefore, we will model this RL as a Partially Observable Markov Decision Process (POMDP).

A POMDP is formally defined as $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \Omega, \mathcal{O}, \gamma \rangle$. Where:

- $\mathcal{S}$ is the set of all possible states of the environment.
- $\mathcal{A}$ is the action space
- $\mathcal{P}$ is the transition probability function $P(s'|s, a)$, representing the probability that taking action a in state s will lead to state s'
- $\mathcal{R}$ is the reward function
- Ω is the set of all possible local observations
- $\mathcal{O}$ is the observation function $P(o|s, a)$, representing the agent seeing observation o given state s and previous action was a
- γ is the discount factor

2.2.1 Reward Function

Liu et al.[1] suggests for these more complex games, a dense reward system should be implemented, giving agents to gain useful feedback on every step.

Each agent i:

$$r_t^{(i)} = r_{\text{time}} + r_{\text{move}} + r_{\text{novel}} + r_{\text{interact}} + r_{\text{share}}$$

Where:

$$r_{\text{move}} = \begin{cases} -0.02, & \text{if } a_t^{(i)} \in \{\text{Up}, \text{Down}, \text{Left}, \text{Right}\} \text{ and } \ell_{t+1}^{(i)} = \ell_t^{(i)}, \\ 0, & \text{otherwise.} \end{cases}$$

$$r_{\text{novel}} = 0.002\, \mathbb{I}_{\text{new}}(\ell_{t+1}^{(i)})$$

$$r_{\text{interact}} = \begin{cases} 0.5\, \mathbb{I}_{\text{found}}^{(i)} + 0.7\, \mathbb{I}_{\text{basic}}^{(i)} + 1.0\, \mathbb{I}_{\text{combine}}^{(i)}, & \text{if } a_t^{(i)} = \text{Interact}, \\ 0, & \text{otherwise.} \end{cases}$$

$$\text{FOUND_REWARD} = 0.5, \quad \text{BASIC_REWARD} = 0.7, \quad \text{COMBINE_REWARD} = 1.0$$

$$\bar{r}_t = \frac{1}{N} \sum_{j=1}^{N} \tilde{r}_t^{(j)}$$

$$r_t^{(i,\text{final})} = (1 - \alpha)\, \tilde{r}_t^{(i)} + \alpha\, \bar{r}_t$$

Idea behind rewards

Whats important spare reward problem - how we mitigate credit assignment problem - how we mitigate non stationary problem

Joint Exploration Dilemma Communication Vacuum Equilibrium Semantic Drift

Normalisation of rewards

2.2.2 Multi-Agent Proximal Policy Optimisation(MAPPO)

TODO Citations for MARL problems

The focus of this project is on communication in multi-agent reinforcement learning. However, MARL has many challenges which we are likely to uncover such as the non-stationary problem, credit assignment problem and long training times. The non-stationary problem is when multiple agents learn simultaneously creating an unpredicatably changing environment from an individual agent's perspective. To mitigate these problem, so we can focus our investigation on the communication aspect, we will adjust the original AI Mother Tongue[1] algorithm by replacing their chosen reinforcement learning algorithm of REINFORCE[2] to one which focuses on stability. We will use Multi-agent Proximal Policy Optimisation (MAPPO)[3] - an extension of an approximation of trust region policy optimisation (TRPO) for multiple agents. I have chosen MAPPO in hopes to reduce training times as it only requires a single critic and all agents can share the majority of paramaters, and the global critic can overcome the non-stationary problem and credit assignment problem as argued in [3].

MAPPO optimises two main components. Each agent's Policy (Actor) π_{θ_i} and the Value Function (Critic) V_ϕ.

Policy Objective (Actor)

The goal of the policy is to maximise the clipped surrogate objective, preventing drastic changes to the policy from single timesteps. For every agent i, define the probability ratio $r_i(\theta)$ as:

$$r_i(\theta) = \frac{\pi_\theta(a_i|o_i)}{\pi_{\theta_{old}}(a_i|o_i)}$$

The MAPPO objective function is:

$$L(\theta) = \mathbb{E}_i \left[\min \left(r_i(\theta)\hat{A}_i,\ \text{clip}(r_i(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_i \right) \right]$$

Where:

- $\hat{A}_i$ is the Advantage Estimation
- ϵ is a hyperparameter that limits the policy update

Value Function (Critic)

Unlike standard Proximal Policy Optimisation (PPO)[4], the Value Function is not per-agent, but per-world. It has access to global information. The goal of the critic is minimise the Mean Squared Error between its own prediction and the discounted returns R_t.

$$L(\phi) = \mathbb{E}_t \left[(V_\phi(S_t) - R_t)^2 \right]$$

Generalised Advantage Estimation (GAE)

MAPPO uses Generalised Advantage Estimation[5] to balance bias and variance. To calculate, we first find the Temporal Difference (TD) Residual (representing the error in our current estimate of the state value):

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

We then take an exponentially weighted moving average of our estimated advantage over different time horizons.

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}$$

2.3 Protocols

Making my software re-usable and flexible by making communication protocols interchangeable will allow for comparisons between the protocols.

2.3.1 No Communication

To provide a comparable baseline, we will implement a protocol where agents do not broadcast any information to one another.

2.3.2 On-Policy Communication

2.3.3 On-Policy Communication using Codebook

This is a common (TODO: JUSTIFY) modern methodology to facilitate communication between agents. A simple continuous vector being broadcast from each agent adds a lot of noise to the agents' policy. To reduce this noise, we discretise the continuous vector into a set number of codes. (TODO: Maths, citations)

One issue with this method alone is it is susceptible to codebook collapse. This is when the agents' policy only ever output a small subset of these codes. To mitigate this, we will implement dead code revival. (TODO: Maths, citation)

2.3.4 AI Mother Tongue

Vector-Quantised Variational Autoencoder (VQ-VAE)

Initially introduced by van den Oord et al. [6] and re-explained by Liu et al. [1], VQ-VAE is a generative model that differs from the standard Variational Autoencoder (VAE) by using a quantised latent space.

1. **Components** It is built from three components:

   (a) **Encoder**: Maps input x to a continuous latent representation z_e

   $$z_e = Enc_{\theta_E}(x)$$

   (b) **Quantiser**: Quantises the continuous latent vector z_e into a discrete symbol $z_q \in C$ where C is a finite set of vectors.

   $$k^* = \arg \min_{k \in \{1,\ldots,K\}} \|\mathbf{z}_e - \mathbf{e}_k\|_2^2.$$

   $$\mathbf{z}_q = \mathbf{e}_{k^*}.$$

   $$\mathbf{z}_q = \arg \min_{\mathbf{e}_k \in \mathcal{C}} \|\mathbf{z}_e - \mathbf{e}_k\|_2^2$$

(c) **Decoder**: Reconstructs the input $\hat{x}$ from the discrete symbol z_q.

$$\hat{x} = Dec_{\theta_D}(z_q)$$

2. **Straight-Through Estimator (STE)**

   An issue of VQ-VAEs, is the argmin operator is non-differentiable. If we were to backpropagate classically, gradients would stop at the quantisation step. To solve this, VQ-VAE uses the Straight-Through Estimator where we simply copy the gradients from the decoder input z_q directly to the encoder output z_e.

3. **Loss Components**

   VQ-VAE Loss Function

$$\mathcal{L} = \mathcal{L}_{\text{recon}} + \mathcal{L}_{\text{codebook}} + \beta\mathcal{L}_{\text{commit}}, \tag{2.1a}$$

   where $sg[\cdot]$ denotes the stop-gradient operator, which prevents gradients flowing through its argument in backpropagation. The components are:

   (a) *Reconstruction loss*:

$$\mathcal{L}_{\text{recon}} = \|x - \hat{x}\|_2^2, \tag{2.2}$$

   Defined as the MSE loss between the input x and the output $\hat{x}$

   (b) *Codebook loss*:

$$\mathcal{L}_{\text{codebook}} = \|\text{sg}[z_e(x)] - e\|_2^2, \tag{2.3}$$

   moves codebook vectors e towards encoder outputs $z_e(x)$.

   (c) *Commitment loss*:

$$\mathcal{L}_{\text{commit}} = \|z_q(x) - \text{sg}[z_e(x)]\|_2^2 \tag{2.4}$$

   commits encoder outputs to the nearest codebook entry, weighted by $\beta = 0.25$.

Stochastically-Quantised Variational Autoencoder (SQ-VAE)

As previously discussed, one major issue of using codebooks is codebook collapse. Although this is less prominent in vector-quantised variational autoencoder method implemented in the previous section than the basic version of the on-policy codebook generation (no dead code revival), it still exists due to the 'hacky' straight-through estimator leading to an approximation (TODO: of wat). To fix this issue, Takida et al. [7] propose Stochastically-Quantised Variational Autoencoder (SQ-VAE), a variational Bayes framework for learning VQ-VAE components.

Instead of using the argmin operator, we calculate a categorical probability distribution over the codebook. The probability of picking code k is based on a Gaussian distribution with a learnable paramater σ.

$$q(z_q = e_k|z_e) \propto \exp\left(-\frac{\|z_e - e_k\|_2^2}{2\sigma^2}\right)$$

We apply self-annealing whilst learning σ^2. Early during training, the probability distribution is wide which helps explore the entire codebook. Later on, σ^2 naturally shrinks, allowing the model to make hard choices. To make this differentiable, we use the Gumbel-Softmax trick. In the forward pass, we force the output to be a strict one-hot vector. In the backward pass, we act as if the output was continuous, following this formula:

$$p_k = \frac{\exp(\frac{(logits)_k + g_k}{\tau})}{\sum_{j=1}^{K} \exp(\frac{(logits)_j + g_j}{\tau})}$$

where:

$$\tau = \max(\tau_{min}, \tau_{old} * annealing_rate)$$

However, this method still uses Euclidian distance, but in tasks such as NLP and audio, semantic meaning of a feature is often entirely captured by the direction of the vector, not the magnitude. To improve this method for our use case, we can implement an extension of this model von Mises-Fisher Stochastically-Quantised Variational Autoencoder (vMF SQ-VAE). vMF SQ-VAE uses cosine similarity instead of Euclidian distance. This is done by first normalising the variables onto a unit sphere.

$$z_{norm} = \frac{z_e}{\|z_e\|_2}, \quad e_{norm,k} = \frac{e_k}{\|e_k\|_2}$$

Then using the Gumbel-Softmax trick again but with cosine similarity.

$$q(z_q = e_k | z_e) = \frac{\exp(\kappa \cdot z_{norm}^T e_{norm,k})}{\sum_j \exp(\kappa \cdot z_{norm}^T e_{norm,j})}$$

Hierarchically-Quantised Variational Autoencoder (HQ-VAE)

Yknow what to explain

Pretraining via Imitation Learning

To further reduce the training time required, Liu et al. [1] suggests initially training both the actor and critic models with supervised learning. Required observations and their associated advantages can be precollected from an expert agent.

Explain expert agent, imitation learning

2.4 Requirements Analysis

2.4.1 Success Criteria

1. Design and implement a simulation to train and demo multi-agent interactions. The simulation should be able to have single runs visualised. The simulation should control the state of every agent and collect data.
2. Implement a functional MAPPO-based multi-agent reinforcement learning (MARL) system capable of learning, given a simulation program with set methods.

3. Develop a communication protocol that allows agents to transmit raw data between each other.
4. Similarly, develop another communication protocol that uses a VQ-VAE to transmit vectors between agents.
5. The training and evaluation pipeline should be able to train and evaluate models automatically and allow interchangeably using both communication protocols.

2.4.2 Extension

- Implement a third communication protocol using a hierarchically quantised variational autoencoder (HQ-VAE). This method was suggested by the AI Mother Tongue paper as a possible future work.
- Add hyperparameter tuning for my discrete communication protocol. This can involve the language size, quantisation levels, and message frequency to optimise agent coordination and learning efficiency.
- Add a new task where agents must solve for objectives other than an assembly line (e.g., coordinated navigation, resource gathering, or adversarial avoidance), enabling analysis of communication protocol effectiveness in more diverse MARL settings.

2.4.3 MoSCoW Analysis

Table 2.1: MoSCoW Analysis of project requirements and associated risks.

Priority	Feature	Risk
Must Have		

2.5 System Design & Tool Selection

2.5.1 Software Development Model

2.5.2 Tool Selection & Justification

Python was chosen as the programming language for the controller, primarily due to its excellent library support for machine learning and data analysis (including but not limited to NumPy ¬R, Pandas ¬R, Matplotlib ¬R and Jupyter Notebooks ¬R). PyTorch ¬R was chosen to develop the models due to its high performance and imperative execution model, which facilitates rapid iterative prototyping. However, Python is not a performance language and, as of development ¬R, has a GIL ¬R meaning executing simulations of my environment must be effectively done serially, bottlenecking my performance heavily. I chose Rust to develop my environment for its high performance and memory safety allowing for rapid development.

To streamline my package management and environment integration to the controller, I used a pyproject.toml file. This file handled installing packages from external sources and also building my rust environment program to allow my Python controller to access the environment.

For version control, I used GitHub to store my git repository which included both my program and my dissertation. I also regularly updated the version of code on my laptop as an extra backup.

2.6 Starting Point

I have extensive prior experience using Python. My internship over the previous summer introduced me to the Rust programming language and using the Py03 library to work with both Rust and Python. I had no prior experience with PyTorch, however I understood the fundamentals of neural networks from Cambridge courses and self instructed exercises such as creating a neural network from scratch to solve XOR and using genetic algorithms on the game Snake.

Chapter 3

Implementation

Contribution to the field, with genuine potential for impact outside the tripos. Challenging goals and substantial deliverables, all methods and tools deployed expertly. Original techniques or methodologies going beyond what was previously known. Presentation is clear and concise throughout, with creative use of figures or diagrams. Excellent repository overview, giving clear insight into project structure.

Summary

3.1 Implementation Overview

3.2 System Layout

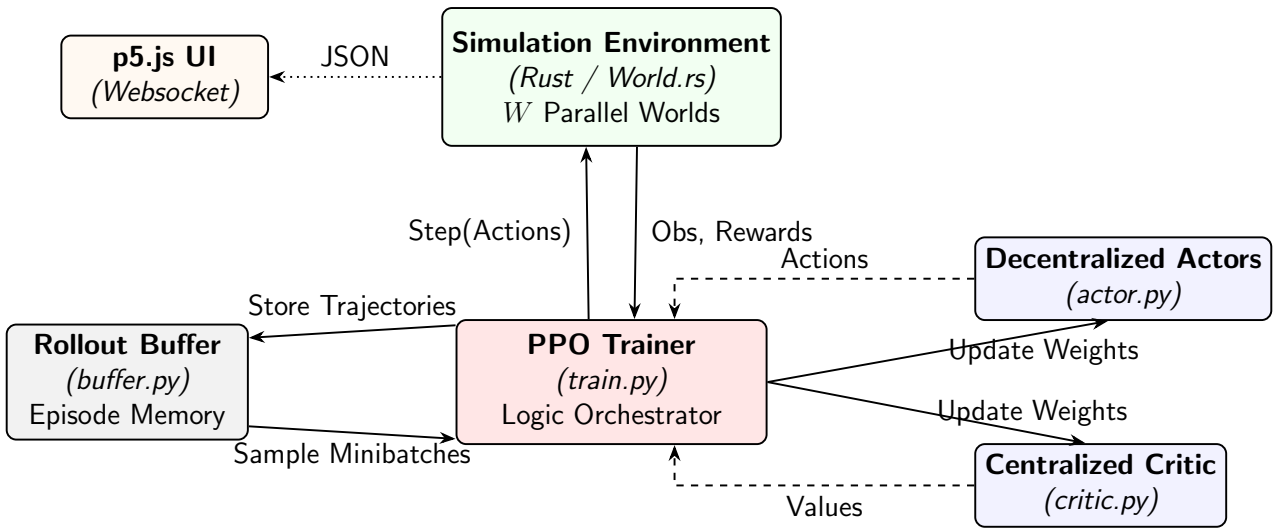

Figure 3.1: System Architecture Diagram: The PPO Trainer orchestrates the rollout phase with the Rust environment and the optimization phase using data sampled from the Rollout Buffer.

3.3 Repository Overview

3.4 Implementation Approach

3.4.1 Unit Testing

3.4.2 Documentation

3.5 Simulation

3.5.1 Environment Logic

3.5.2 Observations

3.5.3 Rewards

3.6 PPO Architecture

3.6.1 Actor-Critic Method

3.6.2 Vectorization and Scaling

3.7 Communication Channel

3.7.1 Continuous Baseline

3.7.2 Discrete VQ-VAE

3.7.3 Linguistic Metrics Implementation

3.8 Summary

Chapter 4

Evaluation

Compelling argument demonstrating success criteria met, or well-justified explanation of different direction taken. Excellent critical thought and interpretation which substantiate any claims of success.

4.1 Hmm

Chapter 5

Conclusions

Conclusions provide a clear and thorough summary, with interesting future work suggested. Thoughtful personal reflection on the lessons learned.

5.1 yep

Bibliography

[1] Hung Ming Liu. "AI Mother Tongue: Self-Emergent Communication in MARL via Endogenous Symbol Systems". In: *arXiv preprint arXiv:2507.10566* (2025). URL: https://arxiv.org/abs/2507.10566.

[2] Ronald J. Williams. "Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning". In: *Machine Learning* 8 (1992), pp. 229–256. DOI: 10.1007/BF00992696.

[3] Chunxu Yu et al. "The Surprising Effectiveness of MAPPO in Cooperative Multi-Agent Games". In: *arXiv preprint arXiv:2301.08454* (2023).

[4] John Schulman et al. *Proximal Policy Optimization Algorithms*. 2017. eprint: arXiv:1707.06347.

[5] Shivang Shrivastav. *Generalized Advantage Estimation (GAE): A Deep Dive into Bias, Variance, and Policy Gradients*. Medium. Accessed: 2026-04-02. Mar. 2025. URL: https://shivang-ahd.medium.com/generalized-advantage-estimation-a-deep-dive-into-bias-variance-and-policy-gradients-a5e0b3454dad.

[6] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. *Neural Discrete Representation Learning*. 2017. eprint: arXiv:1711.00937.

[7] Yuhta Takida et al. *SQ-VAE: Variational Bayes on Discrete Representation with Self-annealed Stochastic Quantization*. 2022. eprint: arXiv:2205.07547.

Appendix A

Proposal

Whisperwise: Emergent Communication in Multi-Agent Reinforcement Learning for Complex Tasks

Part II Computer Science Project Proposal

April 5, 2026

Project Supervisor: **Ian Lewis**
Director of Studies: **Ramsey Faragher**

A.1 Introduction

Since Google DeepMind released AlphaGo in 2016, there has been a steady increase in interest in the field of reinforcement learning. This has led to breakthroughs in areas of finance, trading, robotics and even video games. A clear next step for reinforcement learning is multi-agent reinforcement learning, where multiple agents are placed into the same environment and have to learn to interact with one another efficiently. This area of research has the potential to improve surgery robots, traffic optimisations and search and rescue operations. However, there exists a range of challenges which make multi-agent reinforcement learning more difficult in practice, such as: non-stationary problem, credit assignment issues, and the one I will focus on in this project, communication problems. Unlike single-agent reinforcement learning, multi-agent reinforcement learning can communicate to improve the agent's knowledge of the environment and/or teach them the optimal next step. A widespread issue with communication is the "Joint Exploration Problem", which is a struggle for agents to communicate effectively in novel environments.

I propose to extend AI Mother Tongue[1] to a realistic situation for a more rigorous evaluation of the paper's proposed method. I will simulate an assembly line for multiple agents to evaluate the communication protocols. I will train the agents using a Value Decomposition Network to work together to explore the area. I will have a continuous communication protocol which broadcasts raw information from agent to agent and use this as a baseline to compare to my emergent communication protocol. This other protocol will use a vector quantised variational autoencoder (VQ-VAE) to generate discrete symbols to broadcast from agent to agent.

This new method, using a VQ-VAE, allows communication protocols to emerge naturally rather than relying on manually designed rules or incentives. The limited alphabet forces agents to communicate with more compact, informative, and interpretable messages that prioritise essential information sharing to achieve coordination. Hence, we expect an improvement over the baseline communication protocol in the time taken to explore a given environment.

A.2 Structure of the Project

There are five main components to this project:

- Simulation
- Value Decomposition Network
- Communication Protocols
- Training
- Evaluation: refer to the Evaluation section

Simulation

The simulation is required as it will be how the reinforcement learning algorithm is both trained and evaluated. It will be responsible for the state management and local observations of each agent. Interactions will be handled through discrete timesteps, where agents produce actions informed by their policy and their communication inputs. As training multi-agent systems will require a lot of runs, the simulation must be optimised highly. It will be written in Rust for this reason and mainly function as a headless program (only providing a simple graphical UI when demoing single runs).

The simulation will involve agents working on an assembly line and communicating to pass items along the line.

Value Decomposition Network

The reinforcement learning will optimise the agent's policy (the agent's mapping from state space to action space). I have chosen to use a Value Decomposition Network to facilitate decentralised policy learning, as well as simultaneously optimising for a joint reward. Each agent will maintain its own Q function.

We want the agents to learn coordinated behaviour but maintain their own local observations and local policy. We also want to use a global reward function (successful items assembled). Due to these factors, I have chosen to use a Value

Decomposition Network (VDN). In VDN, the value function of the entire team, Q_{tot}, is factorised as the sum of the individual agents' value functions:

$$Q_{\text{tot}}(\tau, a) \;=\; \sum_i Q_i(\tau_i, a_i)$$

where Q_i is the action-value function for agent i, based only on its own action a_i and local action-observation history τ_i.

The VDN framework is particularly suitable for my project's cooperative multi-agent tasks:

- It naturally encourages cooperation by tying all agents' learning to the same joint reward.
- It is lightweight and efficient compared to more complex mixing architectures (like QMIX), allowing me more time to focus on other parts of the project.
- The decomposition aids in the interpretability of learned policies, as each agent's value estimates can be analysed independently.

Planned Implementation:

- Each agent will be equipped with its own Q-network, parameterised by a deep neural network, receiving as input both its local observations and any messages broadcast by other agents.
- The value decomposition will be enforced in the training loss, aligning each agent's updates to the global objective.
- This setup will also allow for seamless integration and fair comparison of different communication protocols, as the underlying learning dynamics for coordination are held constant.

Communication Protocols

This is the core of the project. I will primarily produce two distinct communication protocols. A simple protocol which has the goal of providing a baseline for the next protocol. It will emit raw continuous vectors. Our other protocol will emit symbols. These will be produced from a VQ-VAE (Vector Quantised - Variational Autoencoders), providing a discrete alphabet for agents to communicate via. Both of the protocols will initially broadcast every message to every other agent.

Training

The training will manage the model's optimisation process. It will use the simulation to collect data and update both the agents' Q functions and, when required, the VQ-VAE. I will add a simple logging and checkpointing framework to prevent loss of results.

A.3 Evaluation

Quantitive

I will gather a small subset of simulations not yet seen by the training, and run each model with the different communication protocols on each. I will then compare the average time for the agents to assemble all the items, as well as the count of successfully assembled items.

Qualitative

I will generate graphs of the distribution of the messages passed between agents. I will also observe the evaluation runs and note down strategies I see the agents come up with. I will then compare the two models' strategies.

A.4 Starting Point

I have limited experience in using reinforcement learning. However, I have previously built an autoencoder when analysing the stock market, which will help me develop the variational autoencoder. Furthermore, I have a small amount of experience using Rust from a three month internship I completed, but I have strong fluency in Python. This will help my development of a high-performance backend (Rust) and the ML (Python).

A.5 Success Criteria

1. Design and implement a simulation to train and demo multi-agent interactions. The simulation should be able to have single runs visualised. The simulation should control the state of every agent and collect data.

2. Implement a functional VDN-based multi-agent reinforcement learning (MARL) system capable of learning, given a simulation program with set methods.
3. Develop a communication protocol that allows agents to transmit raw data between each other.
4. Similarly, develop another communication protocol that uses a VQ-VAE to transmit vectors between agents.
5. The training and evaluation pipeline should be able to train and evaluate models automatically and allow interchangeably using both communication protocols.

A.6 Extensions

- Implement a third communication protocol using a hierarchically quantised variational autoencoder (HQ-VAE). This method was suggested by the AI Mother Tongue paper as a possible future work.
- Add hyperparameter tuning for my discrete communication protocol. This can involve the language size, quantisation levels, and message frequency to optimise agent coordination and learning efficiency.
- Add a new task where agents must solve for objectives other than an assembly line (e.g., coordinated navigation, resource gathering, or adversarial avoidance), enabling analysis of communication protocol effectiveness in more diverse MARL settings.

A.7 Timetable and Milestones

Weeks 1–2 (20 Oct 2025 – 2 Nov 2025)

- Set up Rust-based simulation environment using PyO3 to integrate with PyTorch.
- Prepare PyTorch environment for VDN.

Weeks 3–4 (3 Nov 2025 – 16 Nov 2025)

- Implement simulation for warehouse assembly line.
- Develop a wrapper for the simulation in Rust with py03 using an abstract class (to help potential later extension of using other tasks).

Weeks 5–6 (17 Nov 2025 – 30 Nov 2025): Core VDN Build

- Implement basic value decomposition networks (VDN).
- Develop basic training loop and agent coordination.
- *Milestone:* VDN architecture operational and agents learning basic tasks.

Weeks 7–8 (1 Dec 2025 – 14 Dec 2025)

- Integrate a continuous communication channel into the agent architecture using a modular design.
- Determine which data each agent should communicate.
- *Milestone:* Continuous communication protocol up and running.

Weeks 9–10 (15 Dec 2025 – 28 Dec 2025)

- Develop a variational autoencoder which quantises the latent space (VQ-VAE)
- Integrate the VQ-VAE into a discrete communication protocol for the agents.
- *Milestone:* Discrete communication protocol up and running.

Weeks 11–12 (29 Dec 2025 – 11 Jan 2026)

- Build a training pipeline supporting both communication protocols.
- Implement regular model checkpoints and experiment tracking.
- Automate logging and result collection for training runs.
- *Milestone:* Training pipeline working.

Weeks 13–14 (12 Jan 2026 – 25 Jan 2026)

- Run training on agents using both discrete and continuous communication protocols.
- Collate performance logs and monitor checkpointed models.
- *Milestone:* Complete initial fully trained models for both communication types.

Weeks 15–16 (26 Jan 2026 – 8 Feb 2026)

- Implement scripts to calculate key metrics: learning efficiency, coordination, bandwidth use.
- Automate model evaluation and result logging.

Weeks 17–18 (9 Feb 2026 – 22 Feb 2026)

- Start on extensions, focusing initially on implementing the alternative discrete communication protocol with an HQ-VAE.

Weeks 19–20 (23 Feb 2026 – 8 Mar 2026)

- Catch up on any uncompleted work or drafts.
- Any remaining time can be used to work on extensions.

Weeks 21–22 (9 Mar 2026 – 22 Mar 2026)

- Evaluate all models (continuous, discrete) and tested extensions on chosen environments.
- Collect and analyse results: success rate, sample efficiency, agent coordination, communication cost.

Weeks 23–24 (23 Mar 2026 – 5 Apr 2026)

- Catch up on any uncompleted work or drafts.
- Any remaining time can be used to work on extensions.

Weeks 25–26 (6 Apr 2026 – 19 Apr 2026)

- Write detailed chapters on implementation and evaluation.
- Create figures, tables, and result summaries.

Weeks 27–28 (20 Apr 2026 – 3 May 2026)

- Revise dissertation draft and submit for supervisor feedback.
- Edit dissertation based on feedback.

Week 29 (4 May 2026 – 17 May 2026)

- Final proofreading and formatting.
- Submit dissertation.
- Reserve leftover time for contingency: extra tests, additional evaluation, or further analysis if needed.
- *Milestone:* Submit dissertation and project code.

A.8 Resource Declaration

I will be using my personal desktop computer (AMD Ryzen 7 3700X, 32GB RAM, NVIDIA RTX 5060 Ti) as my primary machine for software development. As a backup, I will use my personal laptop and resources provided by SRCF (student-run computing facility). I will continuously back up my code and dissertation with Git version control on GitHub.